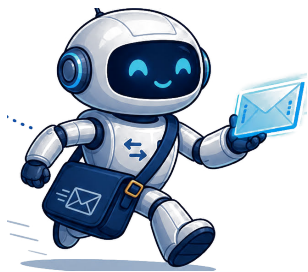
CS 453/553 – Client/Server Systems

Week 1: Foundations of Client/Server Systems

Dr. Dan Schrimpsher

University of Alabama in Huntsville



What Are We Building?

- Real client/server systems
- APIs that real clients can consume
- Systems that communicate over networks
- Code that forces us to think about architecture

We are not just writing programs. We are building systems.

Why Client/Server Matters

- Most modern applications are distributed
- Browsers, mobile apps, cloud services, and databases all communicate remotely
- Local code often depends on remote services
- Client/server is the foundation for web systems, cloud systems, and microservices

Mental Model

Core Idea

Function calls become messages.

That one change affects latency, failure, security, scaling, and design.

A Short History of Distributed Computing

- 1960s: Mainframes and terminals
- 1980s: Personal computers and LAN file servers
- 1990s: Classic client/server applications
- 2000s: Web applications
- 2010s and beyond: Cloud, APIs, microservices, and SaaS

The tools changed, but the core problem stayed the same: separate machines cooperating over a network.

The Big Shift

Old model

One machine runs the whole program.

Distributed model

Multiple machines cooperate across a network.

Function calls become messages.

Monolithic Systems

- One codebase
- One deployment unit
- User interface, business logic, and data access live together

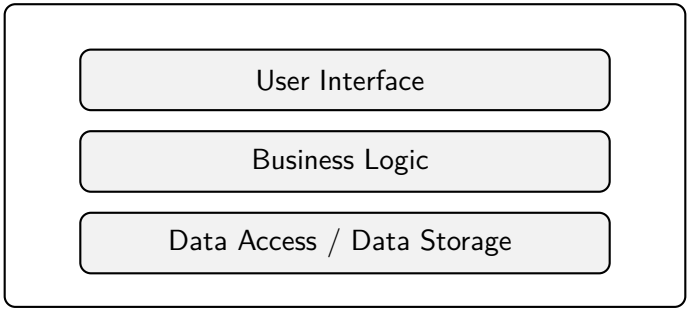
Strength

Simple to start and easy to reason about early.

Weakness

Harder to scale, modify, and deploy independently.

Monolith: Conceptual View

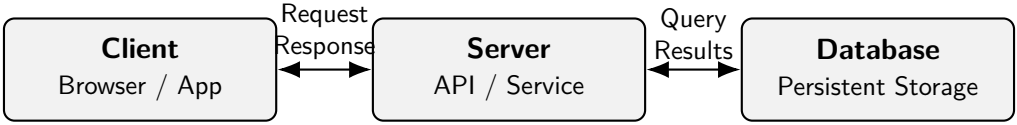


In a monolithic system, the major parts of the application live inside one deployment unit.

Client/Server Systems

- Client requests a service
- Server provides a service
- Responsibilities are separated
- Communication happens across a network boundary

Client/Server: Conceptual View



The client communicates with the server, and the server accesses the database on the client's behalf.

What Is a Client?

- Initiates communication
- Sends requests
- Displays or consumes results

Examples

Browser, mobile app, command-line tool, desktop application

What Is a Server?

- Waits for requests
- Applies logic
- Accesses resources
- Sends responses

Examples

Web API, database server, authentication service, file server

Sockets

- A socket is an endpoint for network communication
- Usually identified by an IP address and port
- Provides the low-level foundation for many protocols

Example

A web server may listen on port 80 or 443.

TCP vs UDP

TCP

- Reliable
- Ordered
- Connection-oriented
- Used by HTTP/HTTPS

UDP

- Lower overhead
- No delivery guarantee
- No connection setup
- Used in streaming, games, DNS

Typical Socket Flow

1. Server starts
2. Server listens on a port
3. Client connects
4. Client sends data
5. Server sends response
6. Connection closes or remains open

Today's Lab: TCP Echo Server

Goal

Build a server that listens on a port.

- **Client:** Connects to the server and sends a message.
- **Server:** Reads the message and sends a response.

Key Idea

The network boundary turns a simple function call into bytes sent over a socket.

Courier-453 delivers packets.

Request/Response Model

- Client sends a request
- Server processes the request
- Server sends a response
- This is the core pattern behind HTTP APIs

HTTP Example

Request

```
GET /users/123 HTTP/1.1
Host: example.com
```

Response

```
{
  "id": 123,
  "name": "Alice"
}
```

Stateless Systems

- Each request contains everything needed
- Server does not remember prior requests
- Easier to scale horizontally
- Common in REST-style APIs

Stateful Systems

- Server remembers context
- Often uses sessions, connections, or stored workflow state
- Can make user workflows easier
- Can make scaling and recovery harder

Stateless vs Stateful

Stateless

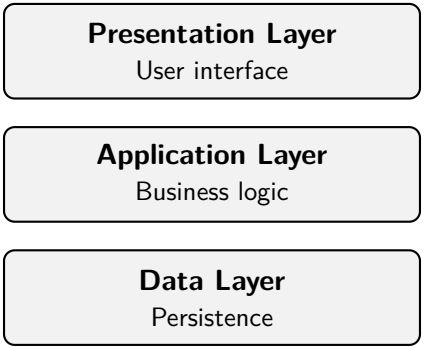
- Server does not remember prior requests
- Each request contains needed context
- Easier to scale horizontally
- Easier recovery after failure

Stateful

- Server remembers context
- Often uses sessions or connections
- More natural for workflows
- Harder to scale and recover

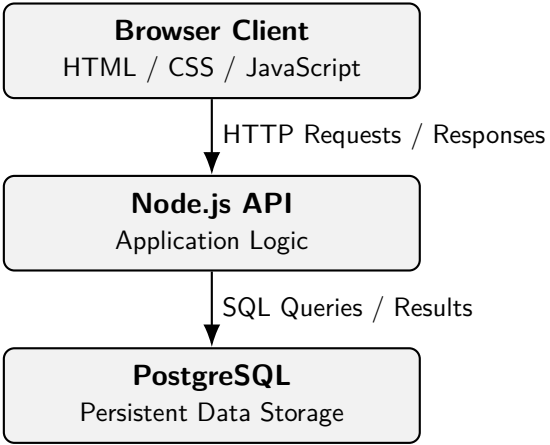
Neither model is always better. The design choice depends on the system's needs.

Layered Architecture



Layers help us manage complexity by separating responsibilities.

Course Architecture



Week 1 Takeaways

- Client/server systems are everywhere
- Network communication changes how we design software
- Sockets are the low-level foundation
- Request/response is the basic web pattern
- Stateless and stateful systems have tradeoffs
- Layers keep systems understandable

Next

- HTTP in more detail
- REST APIs
- First working server
- Browser client talking to backend code